DuocUC | ONLINE



DESARROLLO BACKEND III (PBY2203)

Informe técnico · Evaluación Final Transversal

Modernización del sistema legacy del Banco XYZ

Fernando Fuentes Allende

Índice

Resumen ejecutivo....4

Diagnóstico del sistema legacy....4

Procesos clave de la migración....5

Requerimientos del negocio y propuesta de arquitectura....6

- Tres requerimientos clave....6
- Arquitectura propuesta....7
- Decisiones de arquitectura y su justificación....7
- Casos de uso....9

Parte 1: migración de procesos batch....10

- Manejo de errores....10
- Paralelismo y escalabilidad....11
- Políticas de finalización y reejecución....11
- Equivalencia con el legacy....12

Parte 2: patrón Backend for Frontend....12

- Optimización del rendimiento....12
- Seguridad por canal....13
- Punto de entrada único....13

Parte 3: microservicios resilientes....13

- Los tres microservicios....13
- Spring Cloud....14
- Seguridad con OAuth2....14
- Resiliencia con Resilience4j....14
- Eventos con Kafka....15
- Consistencia de los datos....16

Monitoreo y registro de operaciones....17
Contenedores, escalabilidad y despliegue en AWS....17
Docker y Docker Compose....17
Escalabilidad horizontal....17
Despliegue en AWS....18
Integración entre los componentes....19
Verificación....19
Comparación con el sistema legacy....20
Desafíos y soluciones....21
Propuestas de mejora y próximos pasos....22
Conclusiones....22
Referencias....23

Resumen ejecutivo

El Banco XYZ opera desde hace más de treinta años sobre una plataforma legacy en COBOL y scripts Shell que corre en mainframe. Esa plataforma es costosa de mantener, no escala y dificulta integrar tecnologías nuevas. Este informe documenta la solución desarrollada para modernizarla: una arquitectura de microservicios, empaquetada en contenedores y lista para desplegarse en la nube.

La solución cubre las tres partes del encargo dentro de un mismo repositorio, que se construye y se levanta con un solo comando:

- **Parte 1. Procesos batch.** Los tres procesos nocturnos del legacy (reporte de transacciones diarias, cálculo de intereses y estados de cuenta anuales) se reescribieron como Jobs de Spring Batch, con manejo de errores, procesamiento en paralelo y reejecución automática.
- **Parte 2. Backend for Frontend.** Cada canal (web, móvil y cajero automático) tiene su propio backend, con su autenticación, su formato de respuesta y su certificado HTTPS.
- **Parte 3. Microservicios resilientes.** El monolito se dividió en tres microservicios con base de datos propia (clientes, cuentas y pagos), sobre Spring Cloud, protegidos con OAuth2, tolerantes a fallos con Resilience4j y comunicados por eventos a través de Apache Kafka.

El sistema completo son doce aplicaciones Spring Boot más PostgreSQL y Kafka, cada una en su contenedor Docker. El mismo archivo de orquestación sirve para ejecutarlo en un computador, en integración continua y en una instancia de AWS, donde los tres microservicios de negocio se escalan a varias réplicas sin detener el servicio.

Diagnóstico del sistema legacy

El caso describe tres problemas, uno por cada parte del encargo. La tabla los resume junto con la consecuencia que tienen para el banco.

Ámbito	Situación actual	Consecuencia para el banco
Procesos batch	Tres procesos secuenciales en COBOL y Shell. Los archivos de entrada traen errores: fechas en varios formatos, montos negativos o vacíos, tipos inválidos y duplicados.	Un registro inválido puede detener un proceso completo. Ante una falla hay que repetir todo. Los errores llegan a los reportes.
Canales	Web, móvil y cajeros se conectan al mismo backend monolítico y reciben los mismos datos.	Sobrecarga innecesaria en canales con poco ancho de banda. Los equipos de frontend dependen de los tiempos del backend.

Ámbito	Situación actual	Consecuencia para el banco
Arquitectura	Un monolito con seguridad única y centralizada.	Solo escala comprando un mainframe mayor. Una falla en un módulo afecta a todos. El riesgo de seguridad se concentra en un punto.

Procesos clave de la migración

Migrar a microservicios no es un único cambio, sino cinco procesos que se apoyan entre sí. Se identificaron los siguientes, y cada uno quedó materializado en módulos concretos del repositorio.

N.º	Proceso clave	En qué consiste	Dónde se implementó
1	Migración de procesos batch a Spring Batch	Reescribir los tres procesos legacy como Jobs con lectura, procesamiento y escritura por lotes, tolerantes a datos erróneos y reanudables.	Módulo batch
2	División del monolito en microservicios	Separar el sistema por capacidad de negocio. Cada servicio es dueño de sus datos y se despliega por separado.	clientes-service, cuentas-service, pagos-service
3	Implementación del patrón BFF	Dar a cada canal un backend propio que adapta los datos y la seguridad a sus necesidades.	bff-web, bff-mobile, bff-atm y api-gateway
4	Seguridad distribuida con Spring Security y OAuth2	Reemplazar la seguridad central por tokens que cada servicio valida por su cuenta, con permisos mínimos por componente.	auth-server y la configuración de seguridad de cada servicio
5	Mensajería asíncrona con Apache Kafka	Comunicar los servicios mediante eventos, para desacoplarlos y coordinar operaciones que abarcan más de un servicio.	pagos-service, cuentas-service, clientes-service y notificaciones-service

El orden importa. La división en microservicios (2) obliga a resolver la seguridad entre ellos (4) y la consistencia de las operaciones que cruzan servicios, que se resuelve con eventos (5). El patrón BFF (3) protege a los canales de esa división: un frontend sigue haciendo una sola llamada aunque detrás haya tres servicios. La migración batch (1) es independiente del resto y por eso pudo abordarse primero.

Requerimientos del negocio y propuesta de arquitectura

Tres requerimientos clave

El caso pide que la nueva plataforma cumpla con escalabilidad, seguridad y eficiencia operativa. De ahí se desprenden los tres requerimientos que guiaron cada decisión de diseño.

Requerimiento del negocio	Qué exige en concreto	Cómo responde la arquitectura
1. Escalabilidad y adaptación a la demanda	Atender más carga agregando capacidad, sin detener el servicio ni reemplazar el hardware.	Microservicios sin estado en memoria, con base de datos propia; réplicas que se registran solas en Eureka; balanceo de carga en el gateway y en cada cliente; particiones de Kafka repartidas entre réplicas; contenedores Docker.
2. Seguridad en un entorno distribuido	Proteger los datos y el dinero cuando ya no existe un único punto de control.	OAuth2 entre servicios con un permiso (scope) por microservicio; autenticación propia de cada canal; HTTPS en cada canal; los microservicios no son alcanzables desde fuera de la red interna.
3. Continuidad operativa y consistencia de los datos	Que la falla de un componente no detenga al resto, y que el dinero nunca se pierda ni se duplique.	Circuit breaker, reintentos y respuestas alternativas con Resilience4j; saga con compensación para las transferencias; operaciones idempotentes; procesos batch que omiten registros inválidos y se reanudan solos.

Arquitectura propuesta

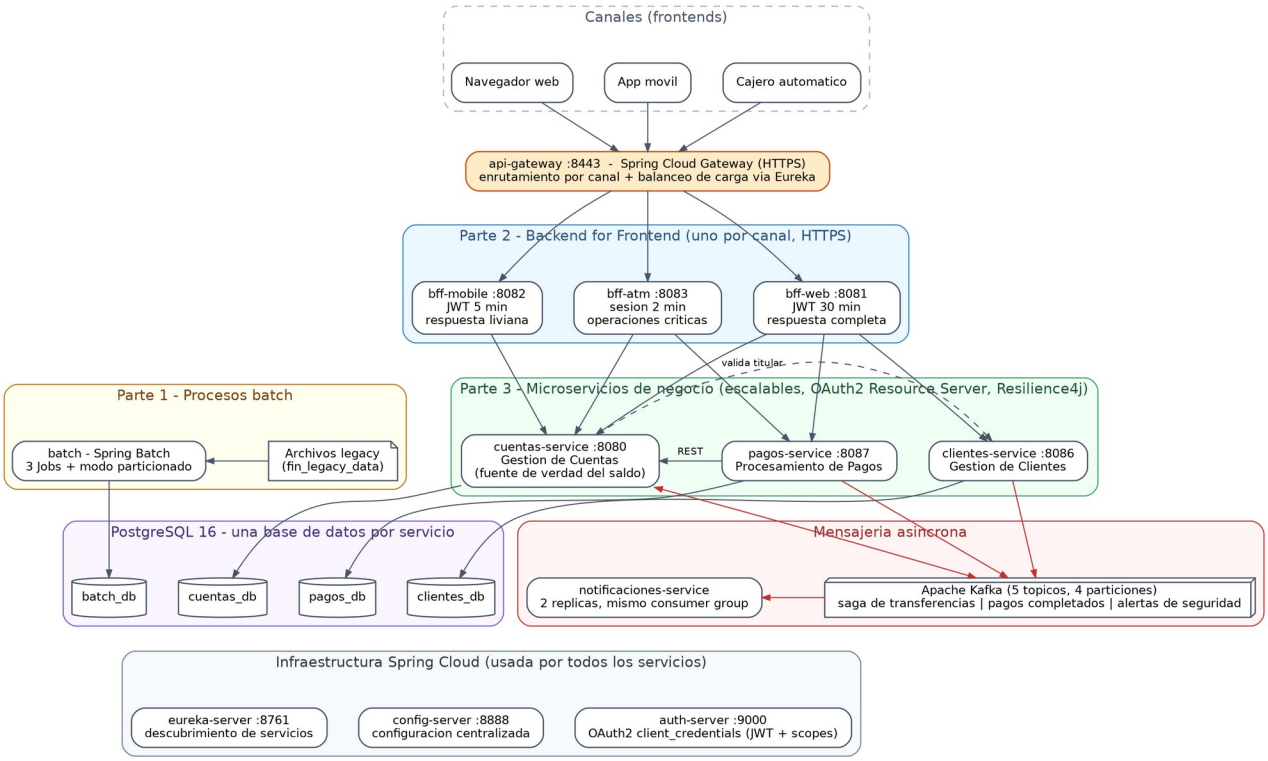


Figura 1. Arquitectura general del sistema.

La arquitectura se organiza en capas. Los canales entran por un único punto, el **api-gateway**, que enruta cada petición al BFF de su canal. Cada **BFF** autentica al usuario, consulta los microservicios que necesita y arma una respuesta a la medida del canal. Los tres **microservicios de negocio** son los únicos que acceden a los datos, cada uno a su propia base. Las operaciones que involucran a más de un servicio se coordinan con eventos en **Kafka**. Tres servicios de infraestructura dan soporte a todos: **config-server** entrega la configuración, **eureka-server** permite que los servicios se encuentren y **auth-server** emite los tokens con que se autentican entre sí. Los **procesos batch** corren aparte, a pedido, sobre su propia base de datos.

Decisiones de arquitectura y su justificación

Decisión	Alternativa descartada	Justificación
Una base de datos por microservicio (PostgreSQL)	Base de datos compartida, o datos en memoria	Es la condición para escalar: todas las réplicas de un servicio ven el mismo estado, y ningún servicio depende del esquema de otro. Con datos en memoria, cada réplica tendría saldos distintos.
Saga por coreografía con Kafka para las transferencias	Transacción distribuida, u orquestador central	No existe una transacción que abarque dos bases de datos. Cada paso es una transacción local, y un paso fallido se repara con una compensación explícita. No hace falta un componente coordinador adicional.

Decisión	Alternativa descartada	Justificación
REST sincrónico para depósitos y pagos de servicios	Hacer todas las operaciones asíncronas	Tocan una sola cuenta y el canal espera la respuesta. La llamada se protege con reintentos y circuit breaker, y lleva una clave de idempotencia que hace seguro reintentarla.
OAuth2 con un scope por microservicio	Una clave compartida entre todos los servicios	Mínimo privilegio: si se filtran las credenciales de un canal, el daño se limita a lo que ese canal podía hacer. Se pueden rotar sin afectar a los demás.
Autenticación de usuario propia en cada BFF	Un único inicio de sesión para todos los canales	Cada canal tiene riesgos distintos: sesión larga en la web, corta en el móvil y de un solo uso en el cajero.
Bloqueo de fila en PostgreSQL al mover saldos	Sincronización dentro de la aplicación	Con varias réplicas, el único punto común donde se puede ordenar el acceso a un saldo es la base de datos.
Los microservicios no publican puertos	Publicar cada servicio en el servidor	Seguridad: solo el gateway y los BFF son alcanzables. Escalabilidad: un puerto fijo solo admite un contenedor.
Módulos independientes, sin código compartido	Una biblioteca común de clases	Cada servicio se construye y despliega por separado. El acoplamiento se limita a contratos HTTP/JSON y a eventos.

Casos de uso

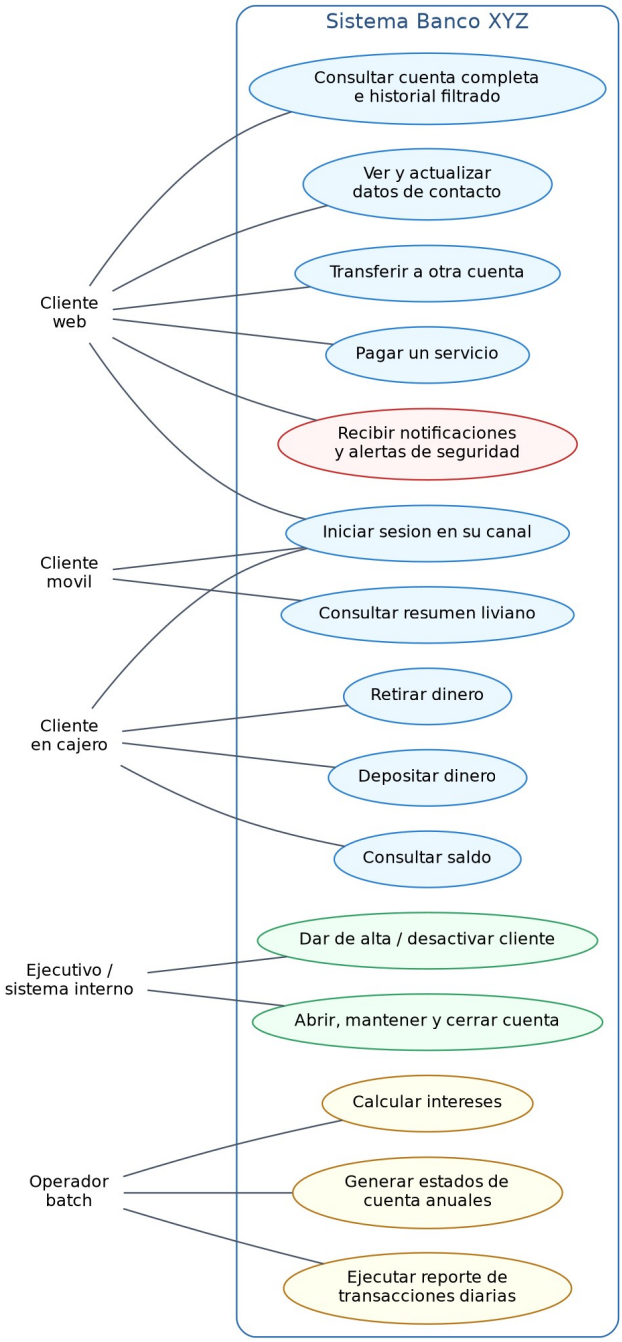


Figura 2. Casos de uso por actor. En azul, los de los canales; en verde, los de gestión interna; en amarillo, los procesos batch.

Parte 1: migración de procesos batch

El módulo **batch** reescribe en Spring Batch los tres procesos del legacy. Lee los archivos del repositorio oficial del caso (fin_legacy_data, 1.000 filas por archivo) y escribe en PostgreSQL.

Job	Archivo de entrada	Qué produce
Reporte de transacciones diarias	movimientos_financieros_diarios.csv	Transacciones validadas, con las anomalías marcadas (monto negativo o cero, tipo no reconocido), y un resumen con totales de créditos y débitos.
Cálculo de intereses	intereses_trimestrales.csv	Interés y saldo final de cada cuenta de ahorro (1,5%) y de préstamo (2,5%).
Estados de cuenta anuales	estados_financieros_anuales.csv	Movimientos corregidos y un estado por cuenta: depósitos, retiros, compras y saldo neto del año.

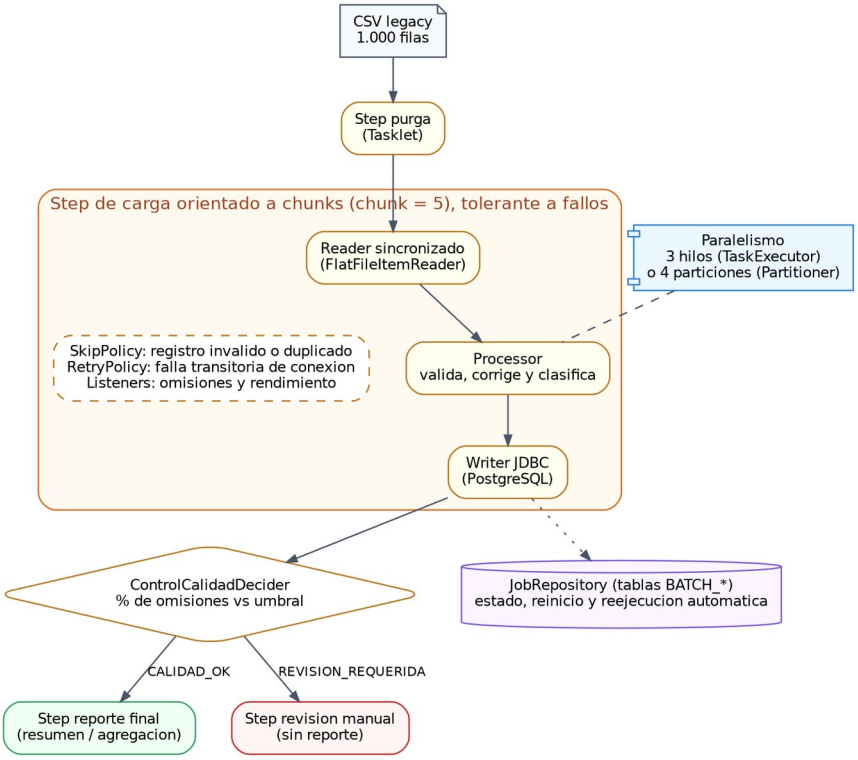


Figura 3. Estructura común de los tres Jobs.

Manejo de errores

Los archivos traen errores intencionales, y el diseño distingue tres situaciones:

- **Datos recuperables: se corrigen.** Las fechas en formatos antiguos se normalizan, una descripción vacía recibe un texto por defecto, un tipo escrito con tilde se unifica y un

monto con el signo equivocado para su tipo se invierte y queda marcado como anomalía.
- **Datos irrecuperables: se omiten.** Un monto vacío, una edad fuera de rango, un tipo de cuenta no soportado o un registro duplicado se saltan mediante una política de omisión propia, y cada uno queda en el log con su motivo. El proceso continúa.
- **Fallas transitorias: se reintenta.** Si la base de datos deja de responder un momento, una política de reintento repite la operación hasta tres veces antes de darla por fallida.

La integridad se refuerza en la base de datos: restricciones de unicidad impiden cargar dos veces el mismo registro, incluso con varios hilos escribiendo a la vez.

Paralelismo y escalabilidad

Hay dos formas de procesar en paralelo, y ambas están implementadas:

- **Multi-hilo.** Los tres Jobs procesan lotes de 5 registros con 3 hilos simultáneos. La lectura está sincronizada; el procesamiento y la escritura, que concentran el trabajo, ocurren en paralelo.
- **Particionado.** El Job de transacciones, el de mayor volumen, tiene una variante que divide el archivo en rangos de filas y ejecuta cada rango como un Step independiente, con su propio estado. Es la base para repartir el trabajo entre varias máquinas.

El tamaño de lote, la cantidad de hilos y la cantidad de particiones se cambian por línea de comandos, sin recompilar. El conjunto de conexiones a la base de datos está dimensionado para cubrir todos los hilos.

Políticas de finalización y reejecución

- **Control de calidad.** Al terminar la carga, un componente de decisión compara el porcentaje de registros omitidos con un umbral. Si lo supera, el Job no genera su reporte y queda derivado a revisión manual: es preferible no entregar un reporte que entregar uno construido sobre un archivo inservible.
- **Límite de omisiones.** Si los registros omitidos superan un tope, el Step falla en vez de seguir perdiendo datos en silencio.
- **Reejecución automática.** Si un Job termina fallido, el lanzador lo vuelve a ejecutar hasta dos veces con los mismos parámetros. Spring Batch reconoce que es la misma ejecución y la reanuda a partir de la metadata que guarda en la base de datos.
- **Código de salida.** El proceso termina con código distinto de cero si un Job no se completó, para que un planificador o un pipeline lo detecten.

Equivalencia con el legacy

Los procesos nuevos producen los mismos resultados que los antiguos para los registros válidos, y agregan trazabilidad para los inválidos. Al aplicar las reglas de validación al dataset oficial se omite el 23,2% de las transacciones diarias y el 16,4% de los movimientos anuales por datos irrecuperables (cifras registradas por Spring Batch en su metadata durante la corrida de verificación). En el archivo de intereses la proporción llega al 78,2%, pero la mayoría no son errores: son cuentas de tipos que el proceso no cubre, como las hipotecarias. Por eso el umbral del control de calidad se fijó en 85%: deja pasar los tres archivos oficiales y sigue deteniendo uno prácticamente inservible.

Parte 2: patrón Backend for Frontend

Se eligió la variante de **backends independientes**: tres aplicaciones separadas, una por canal, que no comparten código. Así el equipo de cada canal puede cambiar y desplegar su backend sin coordinarse con los otros.

	Canal web	**Canal móvil**	**Cajero automático**
Módulo	bff-web	bff-mobile	bff-atm
Necesidad del canal	Interfaz completa, con tablas y filtros	Rapidez y poco consumo de datos	Operaciones críticas, seguras y breves
Respuesta	Titular, tasa, totales e historial completo de movimientos, con filtros por tipo y fecha	Saldo, tipo de cuenta y los tres últimos movimientos	Solo el saldo, o la confirmación de la operación
Funciones	Consulta, perfil del titular, transferencias, pago de servicios e historial de pagos	Consulta resumida	Saldo, retiro y depósito
Autenticación	Cuenta y nombre del titular; token JWT	Cuenta y PIN; token JWT	Tarjeta y PIN; sesión en el encabezado
Duración de la sesión	30 minutos	5 minutos	2 minutos; se invalida tras cada operación
Microservicios que usa	Clientes, cuentas y pagos	Cuentas	Cuentas y pagos

Optimización del rendimiento

Cada BFF pide a los microservicios solo lo que su canal necesita y devuelve solo lo que su canal muestra. El BFF web actúa además como compositor: una sola llamada del navegador se resuelve consultando la cuenta en un servicio y el titular en otro. La respuesta del canal

móvil para una misma cuenta pesa una fracción de la del canal web, porque no incluye el historial completo ni los datos del titular.

Seguridad por canal

- **Autenticación y autorización son controles separados.** Un token válido solo permite operar sobre la cuenta propia: consultar o transferir desde otra cuenta responde con un rechazo.
- **Comunicación cifrada.** Cada BFF y el gateway atienden solo por HTTPS, cada uno con su propio certificado.
- **Mensajes que no revelan información.** Un inicio de sesión fallido responde lo mismo exista o no la cuenta.
- **Reglas propias del canal.** El cajero limita el monto por operación e invalida la sesión después de cada retiro o depósito.

Punto de entrada único

El **api-gateway** (Spring Cloud Gateway) es el único componente expuesto a Internet. Enruta por prefijo de ruta al BFF de cada canal, ubica sus instancias en Eureka y reparte las peticiones entre ellas. Los microservicios no tienen ruta en el gateway.

Parte 3: microservicios resilientes

Los tres microservicios

Microservicio	Responsabilidad	Operaciones	Datos propios
clientes-service Gestión de Clientes	Información personal y perfil de los clientes	Alta, consulta, actualización de datos de contacto, desactivación y reactivación	clientes_db
cuentas-service Gestión de Cuentas	Cuentas y saldos. Es la única fuente de verdad del dinero	Apertura, mantenimiento, cierre, consulta, débito y crédito. Participa en la saga de transferencias	cuentas_db
pagos-service Procesamiento de Pagos	Registro y estado de cada operación de pago	Transferencias, depósitos, pagos de servicios e historial	pagos_db

La división sigue las capacidades del negocio y no las tablas: quién es el cliente, qué cuentas tiene y qué operaciones solicitó son responsabilidades distintas, con reglas y ritmos de cambio distintos. Un cuarto servicio, **notificaciones-service**, solo consume eventos.

Spring Cloud

Requerimiento	Componente	Cómo se usa
Gestión de configuraciones	Spring Cloud Config	La configuración vive en un solo lugar. Un archivo común define lo que debe ser idéntico en todos los servicios (nombres de los tópicos y dirección del servidor de autorización), y cada servicio tiene además el suyo.
Detección dinámica de servicios	Eureka	Cada servicio se registra al arrancar y los demás lo ubican por su nombre. No hay direcciones fijas entre servicios.
Balanceo de carga	Spring Cloud LoadBalancer	Las llamadas entre servicios se reparten entre las instancias registradas.
Enrutamiento	Spring Cloud Gateway	Punto de entrada único, con una ruta por canal.

Seguridad con OAuth2

El **auth-server** (Spring Authorization Server) emite tokens JWT firmados mediante el flujo client_credentials, pensado para la comunicación entre servicios. Cada microservicio es un Resource Server: valida por su cuenta la firma, la vigencia y el emisor de cada token, y exige además su propio scope.

Cliente registrado	cuentas.access	clientes.access	pagos.access	Motivo
bff-web-client	Sí	Sí	Sí	Canal completo
bff-mobile-client	Sí	No	No	Canal de solo consulta
bff-atm-client	Sí	No	Sí	Retiros y depósitos
pagos-service-client	Sí	No	No	Mueve saldos
cuentas-service-client	No	Sí	No	Valida al titular al abrir una cuenta

El resultado es una seguridad distribuida con mínimo privilegio. El token del canal móvil sirve para leer cuentas, pero un intento de usarlo contra el servicio de clientes o el de pagos es rechazado. Tampoco puede pedir un scope que no tiene registrado: el servidor de autorización se lo niega.

Resiliencia con Resilience4j

Cada llamada entre servicios está protegida, y cada protección define qué ocurre cuando la llamada falla.

Llamada protegida	Patrón	Comportamiento alternativo ante la falla
BFF hacia cuentas-service	Circuit breaker	Respuesta controlada de servicio no disponible, con un mensaje claro. Tras cinco fallas el circuito se abre y responde de inmediato, sin intentar la llamada.
bff-web hacia clientes-service	Circuit breaker y degradación	La cuenta se muestra igual, con el titular marcado como no disponible. Un dato accesorio no impide entregar el dato principal.
pagos-service hacia cuentas-service	Reintento y circuit breaker	Tres intentos, que pueden caer en otra réplica. Si todos fallan, el pago queda registrado como fallido con su motivo.
cuentas-service hacia clientes-service	Circuit breaker	La apertura de cuenta se rechaza: no se abre una cuenta para un cliente que no se pudo verificar.
Publicación de eventos en Kafka	Circuit breaker	Si el evento es crítico, la operación se rechaza. Si es informativo, la operación sigue y la falla queda en el log.

Cada dependencia tiene su propio circuito: la caída del servicio de pagos no impide consultar una cuenta. Los errores de negocio, como un saldo insuficiente, no cuentan como fallas ni se reintentan, porque son respuestas correctas de un servicio sano.

Eventos con Kafka

Tópico	Lo publica	Lo consume	Contenido
transferencias.solicitadas	pagos-service	cuentas-service	Inicio de una transferencia
transferencias.debito-aplicado	cuentas-service	cuentas-service	Paso intermedio de la saga
transferencias.resultado	cuentas-service	pagos-service y notificaciones-service	Transacción completada, rechazada o compensada
pagos.completados	pagos-service	notificaciones-service	Depósito o pago completado
seguridad.alertas	pagos-service y clientes-service	notificaciones-service	Transferencia de monto alto; cambio de datos de un cliente

Una transferencia modifica dos cuentas y su registro vive en otro servicio, por lo que no puede resolverse con una sola transacción. Se modela como una **saga**: una secuencia de pasos, cada uno una transacción local, que se comunican por eventos.

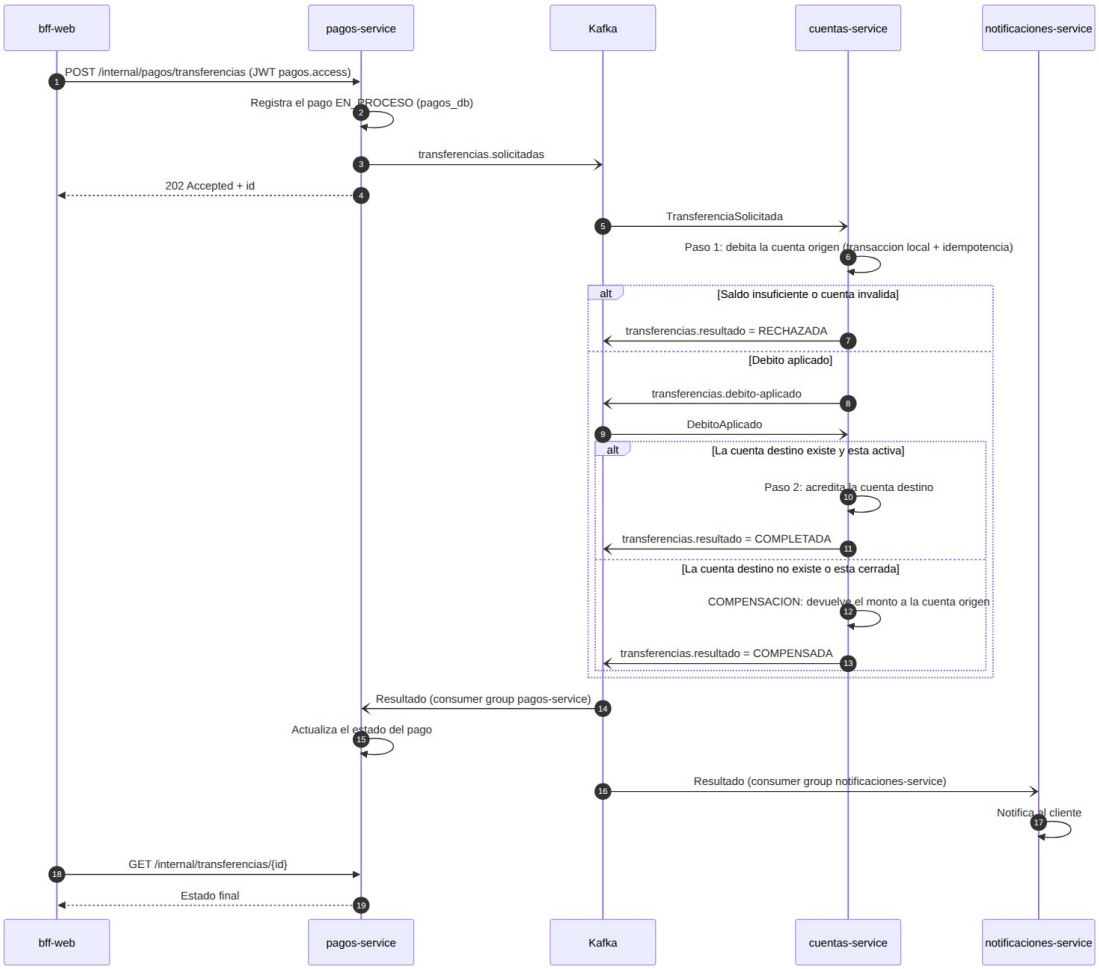


Figura 4. Saga de una transferencia y sus tres desenlaces posibles.

Si la cuenta de origen no tiene saldo, la transferencia se **rechaza** sin haber tocado nada. Si el débito se aplicó pero la cuenta de destino no existe o está cerrada, se ejecuta la **compensación**: el monto vuelve a la cuenta de origen. El canal recibe una respuesta inmediata de solicitud aceptada y consulta después el resultado.

Consistencia de los datos

- **Transacciones locales.** Cada paso que mueve dinero es una transacción de base de datos: o se aplica completo o no se aplica.
- **Bloqueo de fila.** Dos operaciones simultáneas sobre la misma cuenta se ordenan en la base de datos, aunque lleguen a réplicas distintas.
- **Idempotencia.** Kafka puede entregar un evento más de una vez, y un reintento puede repetir una llamada. Cada operación de dinero deja un registro con su identificador en la misma transacción que modifica el saldo; si la operación se repite, se reconoce y no se vuelve a aplicar.
- **Reintento de eventos.** Si un paso de la saga falla por una causa pasajera, se reintenta durante varios minutos antes de descartarlo.

- **Registro previo.** Todo pago se registra como en proceso antes de ejecutarse. Pase lo que pase, queda constancia de que se solicitó y de cómo terminó.

Monitoreo y registro de operaciones

Cada microservicio registra cada petición que atiende: método, ruta, resultado, duración y qué componente la hizo, dato que se obtiene del token. Cada servicio expone además un punto de salud que Docker consulta para saber si está operativo, y métricas a través de Spring Boot Actuator. Los procesos batch guardan en la base de datos el detalle de cada ejecución: registros leídos, escritos y omitidos, duración y estado.

Contenedores, escalabilidad y despliegue en AWS

Docker y Docker Compose

Cada módulo tiene su Dockerfile de dos etapas: la primera compila con Maven y la segunda contiene solo el entorno de ejecución de Java y el archivo empaquetado. La imagen final no incluye el código fuente ni las herramientas de compilación. Un único archivo de Docker Compose define los trece servicios, el orden de arranque según el estado de salud de cada dependencia, los límites de memoria y el reinicio automático de un contenedor que falla. La base de datos se inicializa con un script que crea una base por servicio.

Escalabilidad horizontal

Los tres microservicios de negocio se escalan con un solo comando, con el sistema en marcha. Esto funciona sin cambiar código por cuatro razones:

- No guardan estado en memoria: todas las réplicas comparten la base de datos del servicio.
- Cada réplica se registra sola en Eureka, y los clientes reparten las peticiones entre las registradas.
- Las réplicas comparten el grupo de consumo de Kafka, que reparte entre ellas las cuatro particiones de cada tópico.
- No publican un puerto fijo, de modo que pueden convivir varias en el mismo servidor.

Despliegue en AWS

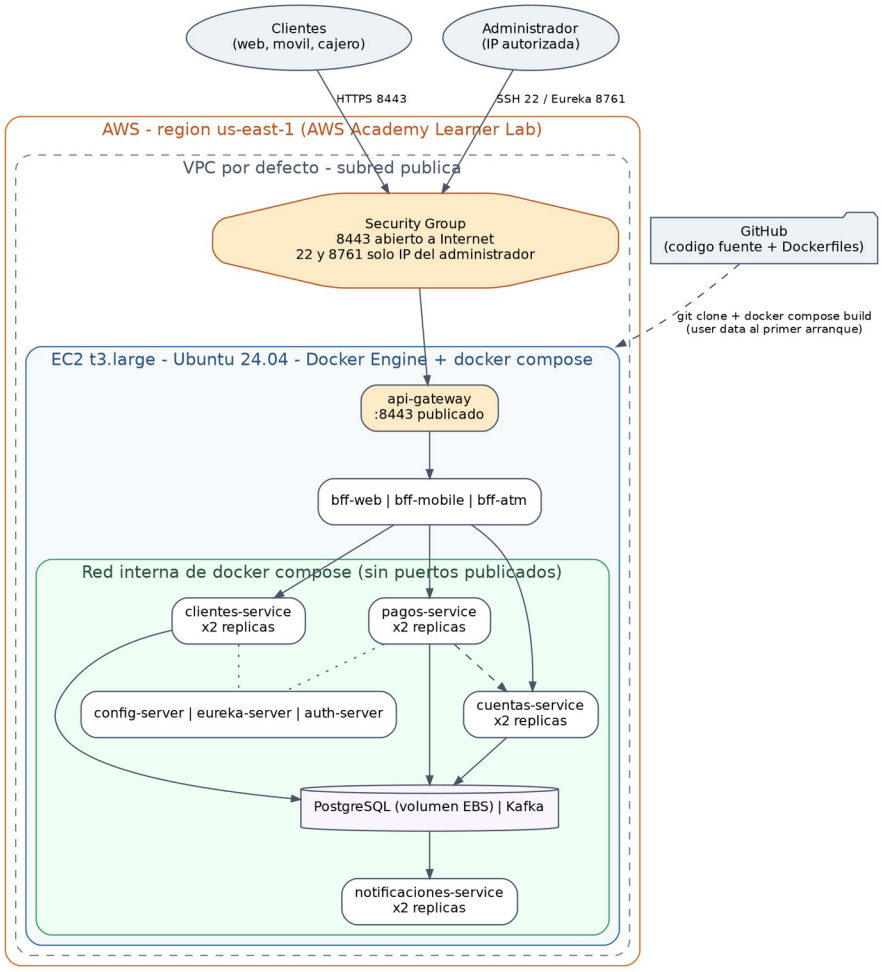


Figura 5. Despliegue en AWS.

El sistema se despliega en una instancia EC2 dentro de AWS Academy Learner Lab. Un script crea el grupo de seguridad y la instancia; al arrancar, la instancia instala Docker, clona el repositorio, construye las imágenes y levanta el sistema sin intervención manual. El grupo de seguridad abre a Internet solo el puerto del gateway; la administración y el panel de Eureka quedan restringidos a la dirección del administrador. Los microservicios, la base de datos y Kafka permanecen en la red interna de Docker.

El despliegue se verificó en una instancia t3.large con Ubuntu 24.04. El script de arranque construyó las doce imágenes y levantó el sistema sin intervención manual en unos quince minutos. Desde un computador externo, a través de Internet, el gateway respondió su chequeo de salud y enrutó el inicio de sesión del canal web hasta su BFF: respondió 200 con un token y con la cabecera que agrega el gateway en cada respuesta.

El documento despliegue.md detalla el procedimiento, la verificación con sus resultados, el escalado en la nube y la evolución hacia un entorno productivo con servicios administrados.

Integración entre los componentes

El recorrido de una transferencia desde el canal web muestra cómo se integran las piezas:

- El navegador llama al **api-gateway**, que ubica al BFF web en **Eureka** y le reenvía la petición por HTTPS.
- El **BFF web** valida el token del usuario y verifica que la cuenta de origen sea la suya.
- El BFF pide un token a **auth-server** y llama a **pagos-service**, a través de un circuit breaker.
- **pagos-service** valida el token, registra el pago en su base de datos y publica la solicitud en **Kafka**.
- **cuentas-service** consume el evento, debita y acredita en su base de datos, y publica el resultado.
- **pagos-service** actualiza el estado del pago y **notificaciones-service** avisa al cliente.
- El navegador consulta el estado y recibe el resultado final.

Todos estos servicios obtuvieron su configuración de **config-server** al arrancar. Los procesos batch se integran por los datos: leen los mismos archivos legacy con que los microservicios cargaron sus bases, y aplican las mismas reglas de validación.

Verificación

El repositorio incluye scripts que ejercitan el sistema de punta a punta y guardan la salida de cada etapa en la carpeta evidencias. Un script los ejecuta todos en orden, partiendo de datos limpios, y el mismo script corre en GitHub Actions con cada cambio que se sube al repositorio. La tabla resume lo que comprueba cada etapa y lo que se obtuvo en la corrida completa sobre Docker, en la que **todas las etapas terminaron correctamente**.

Etapa	Qué comprueba	Resultado obtenido
Procesos batch	Los tres Jobs y el modo particionado; hilos en paralelo; omisiones; control de calidad; reejecución; metadata.	Los tres Jobs terminan COMPLETED sobre 1.000 registros cada uno, repartidos entre 3 hilos (y en 4 particiones de 250 en el modo particionado). Con un umbral de prueba de 20%, el control de calidad deriva el Job a revisión. Con un límite de omisiones forzado, el Job falla y se reejecuta dos veces sobre la misma instancia.
Canales	Operaciones de cada canal; tamaño de las respuestas; autorización; enrutamiento por el gateway.	Para la misma cuenta, la respuesta web pesa 4.984 bytes y la móvil 221. Una cuenta ajena responde 403 y la sesión del cajero, ya usada, 401. El gateway enruta los tres canales (200) y no expone rutas internas (404).

Etapa	Qué comprueba	Resultado obtenido
Seguridad entre servicios	Rechazo sin token; emisión de tokens; mínimo privilegio.	Sin token, los tres microservicios responden 401. El token móvil obtiene 403 en clientes y pagos, y auth-server se niega a emitirle un scope que no tiene registrado (invalid_scope).
Microservicios	Ciclo completo de cliente y cuenta, con validaciones e idempotencia.	Alta, apertura, mantenimiento, depósito, pago, cierre y baja completados. Se rechazan un cliente menor de edad (400), un cliente inexistente (404) y el cierre de una cuenta con saldo (422).
Saga de transferencias	Los tres desenlaces, con saldos antes y después.	Una transferencia COMPLETADA, una RECHAZADA por fondos insuficientes y una COMPENSADA (destino inexistente), con los saldos cuadrados.
Eventos	Consumo de notificaciones y alertas por réplica y partición.	Los mensajes de los cinco tópicos se reparten entre las dos réplicas de notificaciones-service, sin que ninguna procese un evento dos veces.
Escalabilidad	Escalado a dos réplicas con el sistema en marcha y reparto de la carga.	Los tres microservicios pasan a dos réplicas que se registran solas en Eureka; 30 de 30 consultas responden 200 y las peticiones se reparten por igual entre réplicas.
Tolerancia a fallos	Detención deliberada de servicios y recuperación.	Sin clientes-service, la cuenta sigue respondiendo sin datos del titular. Sin cuentas-service, el circuito se abre, el pago queda FALLIDA tras tres reintentos y una transferencia queda retenida en Kafka. Al volver el servicio, el circuito se cierra solo y la transferencia termina COMPLETADA.

Comparación con el sistema legacy

Aspecto	Sistema legacy	Sistema nuevo
Procesos batch	Secuenciales. Un registro inválido detiene el proceso. Ante una falla se repite todo.	En paralelo, con tres hilos o cuatro particiones. Los registros inválidos se omiten y se registran. Se reanuda y se reejecuta de forma automática.
Calidad de datos	Sin control: los errores llegan a los reportes.	Validación, corrección automática y un control que impide generar un reporte sobre datos inservibles.
Canales	Todos reciben la misma respuesta.	Cada canal recibe lo que necesita, con su propia seguridad.
Escalabilidad	Vertical: un mainframe mayor.	Horizontal: más réplicas, sin detener el servicio.
Tolerancia a fallos	Una falla afecta a todo el sistema.	Fallas aisladas, con respuestas alternativas y recuperación automática.
Seguridad	Única y centralizada.	Distribuida, con mínimo privilegio por componente y cifrado en cada canal.
Integración	Acoplada, por archivos y llamadas directas.	Por eventos: se agregan consumidores sin modificar a quienes publican.

Aspecto	Sistema legacy	Sistema nuevo
Despliegue	Manual, sobre el mainframe.	Contenedores reproducibles; el mismo procedimiento en un computador, en integración continua y en la nube.
Costo de cambio	Cualquier cambio exige probar y desplegar todo.	Cada servicio cambia y se despliega por separado.

Desafíos y soluciones

Desafío	Qué ocurría	Solución
Tokens sin permisos	Los servicios rechazaban todas las llamadas con un error de scope insuficiente, que el circuit breaker mostraba como servicio no disponible.	El servidor de autorización no entrega scopes si la petición no los solicita. Se piden de forma explícita en cada solicitud de token. Se aisló el problema llamando al servicio directamente, sin pasar por el BFF.
Estado en memoria al escalar	Con los datos en memoria, dos réplicas del mismo servicio habrían tenido saldos distintos.	Se llevó cada servicio a su propia base PostgreSQL, con bloqueo de fila para los saldos.
Una transferencia entre dos servicios	Dividir el monolito separó el registro del pago y el saldo de las cuentas en bases de datos distintas.	Saga por eventos, con un paso de compensación y operaciones idempotentes.
Entregas repetidas	Kafka puede entregar un evento dos veces, y un reintento puede repetir un débito.	Registro de idempotencia en la misma transacción que modifica el saldo.
Arranque simultáneo de réplicas	Varias réplicas intentando crear las tablas y cargar los datos iniciales a la vez.	El esquema lo crea la base de datos al inicializarse. La carga inicial toma un bloqueo, de modo que solo una réplica la ejecuta.
Demoras del registro de servicios	Un servicio recién levantado respondía, pero los demás aún no lo encontraban.	Arranque ordenado por estado de salud, intervalos de actualización más cortos y scripts que esperan a que el flujo completo responda.
Gateway sano que no enrutaba	El gateway aparecía saludable, pero toda petición a través de él quedaba sin respuesta: el chequeo de salud no pasa por el ruteo.	El log mostró un NoSuchMethodError: la versión de Spring Cloud exigía una versión de Spring Framework más nueva que la incluida en Spring Boot. Se actualizó Spring Boot dentro de la misma línea 3.3 en los doce módulos.
Archivos legacy con muchos errores	Con un umbral de calidad estricto, dos de los tres procesos no generaban su reporte.	Se midió la proporción real de registros omitidos en cada archivo y se calibró el umbral, distinguiendo los errores de los registros fuera de alcance.

Propuestas de mejora y próximos pasos

- **Servicios administrados en AWS.** Pasar de una instancia a un clúster de contenedores (Amazon ECS o EKS) en varias zonas, con base de datos y Kafka administrados (Amazon RDS y MSK). Elimina el punto único de falla y permite escalar de forma automática según la carga.
- **Gestión de secretos.** Llevar las credenciales, hoy valores de demostración en archivos de configuración, a AWS Secrets Manager, y usar certificados emitidos por una autoridad reconocida.
- **Identidad de los usuarios.** Reemplazar el inicio de sesión simplificado por un proveedor de identidad con OpenID Connect y autenticación de dos factores.
- **Publicación confiable de eventos.** Aplicar el patrón outbox para que guardar un pago y publicar su evento ocurran de forma atómica.
- **Observabilidad.** Agregar trazas distribuidas, paneles de métricas y alarmas, y centralizar los logs.
- **Pruebas automatizadas.** Sumar pruebas unitarias y pruebas de contrato entre servicios a las pruebas de punta a punta existentes.
- **Batch en la nube.** Programar los procesos con un planificador y leer los archivos desde almacenamiento de objetos; ejecutar las particiones en máquinas distintas.
- **Precisión monetaria.** Representar los montos de los microservicios con un tipo decimal exacto, como ya lo hacen los procesos batch.
- **Sesiones del cajero.** Llevar las sesiones a un almacén compartido para poder escalar también ese canal.

Conclusiones

La solución reemplaza un sistema monolítico por una arquitectura en la que cada pieza tiene una responsabilidad acotada, puede fallar sin arrastrar a las demás y puede crecer por separado. Los procesos batch dejaron de detenerse ante un dato inválido y pueden reanudarse solos. Cada canal recibe una respuesta a su medida. El dinero se mueve con garantías explícitas de consistencia aun cuando la operación atraviesa varios servicios.

El aprendizaje central es que dividir un monolito traslada la complejidad desde el código hacia la comunicación entre servicios. Las decisiones que más pesaron no fueron las de cada servicio por separado, sino las que definen cómo se autentican entre sí, qué hace cada uno cuando el otro no responde y cómo se mantiene la consistencia sin una transacción común.

Referencias

- Duoc UC (2026). PBY2203 Desarrollo Backend III. Instrucciones específicas y pauta de la Evaluación Final Transversal.
- Villagrán, K. fin_legacy_data. Repositorio de datos del caso. https://github.com/KariVillagran/fin_legacy_data
- Spring. Documentación de referencia de Spring Batch, Spring Cloud, Spring Security y Spring Authorization Server. https://spring.io/projects
- Resilience4j. Documentación. https://resilience4j.readme.io
- Apache Software Foundation. Documentación de Apache Kafka. https://kafka.apache.org/documentation
- Richardson, C. (2018). Microservices Patterns. Manning.
- Newman, S. (2021). Building Microservices (2.ª ed.). O'Reilly.
- Amazon Web Services. Documentación de Amazon EC2. https://docs.aws.amazon.com/ec2



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.